

FINAL PROJECT · ARTIFICIAL INTELLIGENCE

What the market *will ask next*

Forecasting food commodity prices in Cambodia — from World Food Programme price monitoring to a live web app and a Telegram bot.

46 COMMODITIES · 76 MARKETS · XGBOOST, PORTED TO TYPESCRIPT

THE QUESTION

A price is only useful *before you pay it*

The World Food Programme publishes what food cost in Cambodian markets — twenty-three years of it, updated monthly. What it does not publish is what food will cost next month.

That gap is the project: a trader deciding when to buy, a household budgeting for rice, an NGO sizing a food-assistance programme. All of them need the next number, not the last one.

Not a trading signal. A next-period estimate with its uncertainty made visible.

SCOPE

Predict the next reported price

for a given commodity at a given market
— and make that answer reachable in two clicks or one chat message.

THE DATA

Twenty-three years *of market visits*

SOURCE	WFP Global Food Prices — Cambodia (HDX)
OBSERVATIONS	85,107
RANGE	2003-01-15 to 2026-03-15
COMMODITIES	50 raw
MARKETS	86 raw
TARGET	usdprice — price per unit in USD

FILTERING

85,107 rows

everything WFP has recorded



from 2019 onward

older reporting is sparse and pre-dollarisation



commodities with 300+ rows

a series needs history to have a lag



67,654 modelled rows

46 commodities · 76 markets · 4,035 series

The split is the *whole experiment*

Each (commodity, market, pricetype) forms a time series.
The model predicts the next observation in that series
from its own recent history.

THE TRAP

A random train/test split lets the model learn from
June to predict April. Prices are autocorrelated, so it
scores brilliantly and forecasts nothing. The split has
to be chronological.

CHRONOLOGICAL HOLD-OUT

Cut point	2025-04-15 (85th percentile of dates)
Train	57,878 rows · everything before the cut
Test	9,776 rows · everything after
Never seen	the test period, by any model, at any point

Ten features, built *inside each series*

PRICE HISTORY

lag_1

the previous reported price

lag_3

the price three reports back

rolling_mean_3

mean of the last three

CALENDAR

month

1-12, seasonality of harvest

quarter

1-4, coarser seasonality

IDENTITY

commodity_enc

which food

market_enc

which market

admin1_enc

which province

category_enc

cereals, meat, pulses ...

pricetype_enc

retail or wholesale

Lags are computed within each series (grouped shift), never across them — a rice price in Battambang can never leak into a pork price in Phnom Penh. Rows without three prior observations are dropped rather than imputed.

Beating “tomorrow *equals today*”

The naive baseline — predict the last price — already scores R^2 0.92, because prices are sticky. It is the number any model has to beat to be worth deploying.

MODEL	MAE (USD)	RMSE (USD)	R^2
Naive baseline (lag_1)	0.1799	0.4088	0.9239
Random Forest (reproduced)	0.1690	0.3352	0.9489
XGBoost (final model)	0.1642	0.3295	0.9506

Random Forest reproduced locally from the notebook's settings (n=300, depth 15, seed 42).
XGBoost figures come from the trained price_model.pkl itself, evaluated on the held-out period.

RESULT

The model earns *its place*

\$0.164

MEAN ABSOLUTE ERROR

on held-out 2025-26 prices

0.951

R² ON THE TEST PERIOD

variance explained

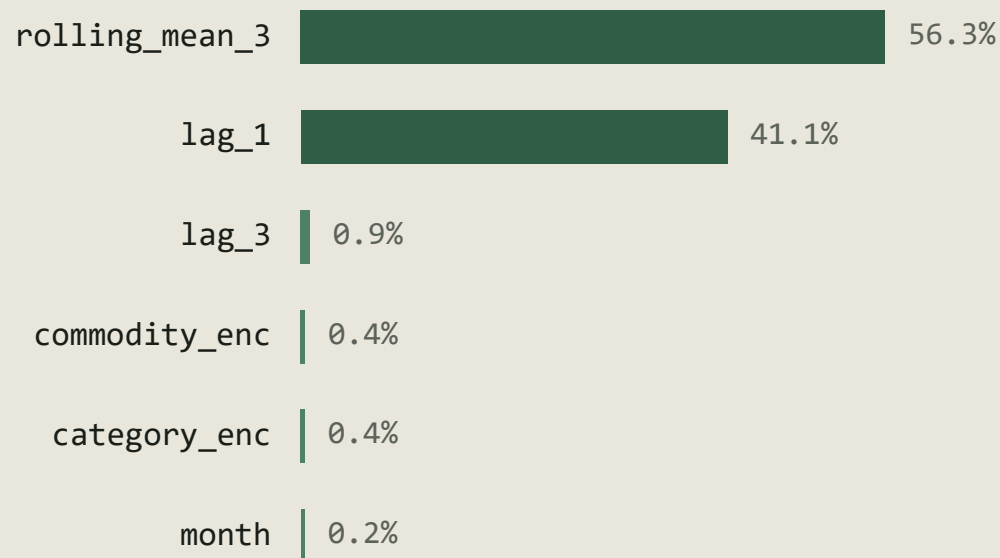
8.7%

BETTER THAN NAIVE

MAE reduction vs. last price

An average miss of about sixteen US cents on a basket whose prices mostly sit between \$0.30 and \$3.00 — accurate enough to plan around, not precise enough to trade on. Both halves of that sentence matter.

It learned to be *a very good smoother*



Gain-based feature importance from the trained booster.

THE HONEST READING

97% of the model

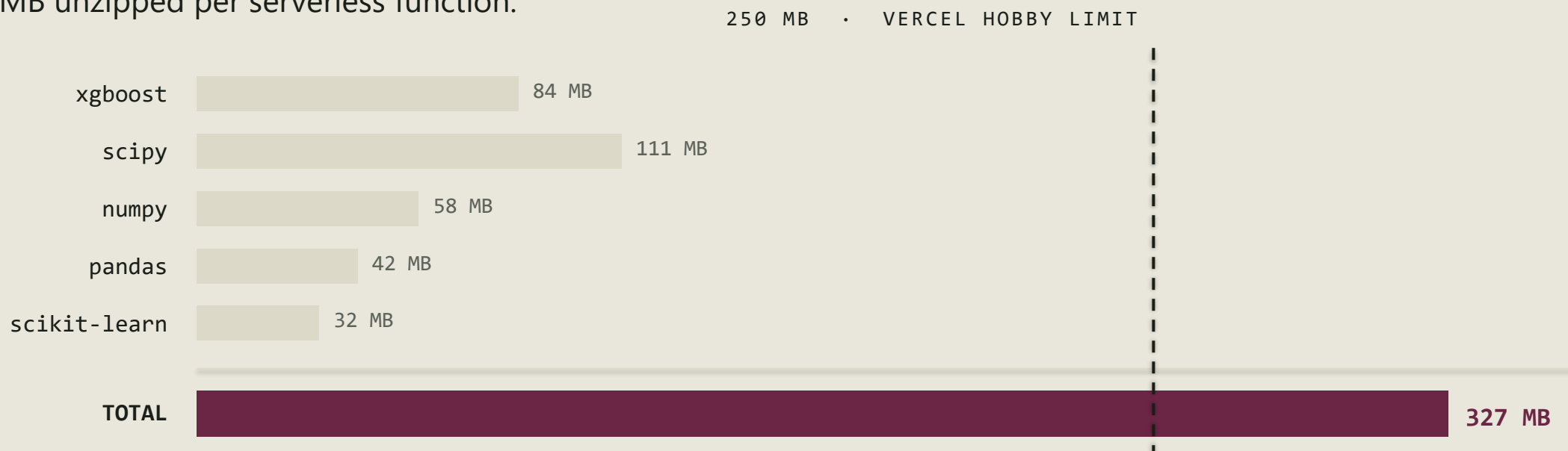
rests on two features: the recent mean and the last price.

Commodity, market, province and season together account for barely two percent. This is a well-calibrated smoother, not a market oracle — and saying so is part of the result.

The model fit.

The libraries didn't.

A notebook that only runs in Colab isn't a product. Deploying to Vercel's free tier meant meeting a hard limit: 250 MB unzipped per serverless function.



The deploy fails before a single prediction is ever served.

A boosted tree is *just a lot of if-statements*

Training needs XGBoost. Predicting does not. Inference is: walk each tree comparing one feature to one threshold, sum the leaf weights, add the base score. Fifteen lines in any language.

400

trees traversed

10

features each

2.4 MB

of JSON

0

ML dependencies

The model exports to JSON: tree arrays, encoder maps, feature order, and a 24-point price history per series. The deployed function ships none of the Python stack.

```
function rawPredict(features: number[]): number {
  let total = BASE_SCORE
  for (const tree of TREES) {
    let node = 0
    while (tree.l[node] !== -1) {
      node = features[tree.f[node]] < tree.t[node]
        ? tree.l[node]
        : tree.r[node]
    }
    total += tree.w[node]
  }
  return total
}
```

Two languages, *one number*

A port is only worth anything if it agrees with the original. The test suite pins the TypeScript traversal to the Python model's own answer for a known series — a wrong feature order or a flipped comparison changes that number, and nothing else would catch it.

19 tests cover the traversal, the fuzzy matching, the commodity/market splitter and every failure path.

REFERENCE CASE

commodity	Rice (mixed, low quality)
market	Phnom Penh · Wholesale
last recorded	2022-06 · \$0.44

python	xgboost	0.460955
--------	---------	----------

typescript	port	0.4609556
------------	------	------------------

agree to 1e-5 · asserted on every test run

THE RESULT

It runs *in a browser*

Nuxt 4 on Vercel. The default forecast is server-rendered, so the chart is on screen at first paint — no spinner, no empty state.

Dependent dropdowns

picking a commodity narrows the markets to those that carry it

Fuzzy input

"rice" resolves to Rice (mixed, low quality) — shortest match wins

Cached at the edge

options for a day, forecasts for an hour

Responsive to 390px

the chart switches to a narrower viewBox, not smaller type

WFP PRICE MONITORING · CAMBODIA

What the market *will ask next*

Next-period price forecasts for **46** commodities across **76** markets, from a gradient-boosted model trained on World Food Programme observations.

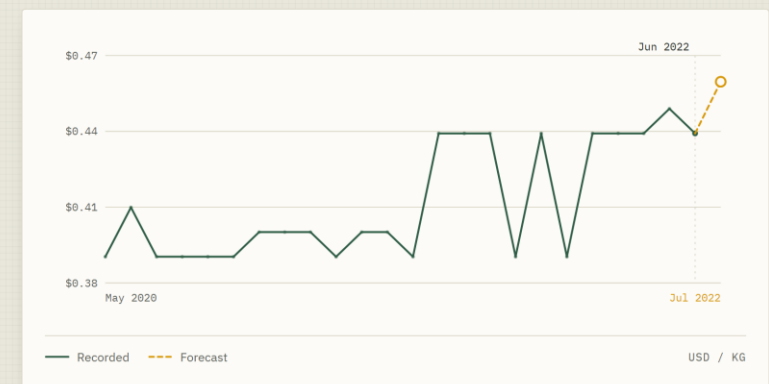
COMMODITY

Rice (mixed, low quality) ▼

MARKET

Phnom Penh ▼

FORECAST



Forecasts are model estimates, not official prices. Each one projects a single reporting period past the last genuine observation for that commodity and market.

The chart ends *in June 2022*

The dataset runs to March 2026. This series does not. Rice wholesale prices in Phnom Penh stopped being reported in June 2022, and the app says so instead of carrying the line forward to look current.

Interpolating to today would have made every screenshot look better and every number less true. Of 4,035 series in the data, many stopped reporting years apart — a single “latest” date across the app would be fiction.

PHNOM PENH, PHNOM PENH · WHOLESALE		
Rice (mixed, low quality)		
LAST RECORDED · JUN 2022	FORECAST · JUL 2022	CHANGE
\$0.440	\$0.461	▲ +4.8%

The same engine, *in a chat window*

One prediction engine, two front doors. The bot shares the exact code the web app calls — a Telegram update in, a formatted forecast out.

Reply in the response body

the webhook returns the sendMessage call itself — no outbound request, and the bot token never reaches Vercel

Always HTTP 200

a non-2xx makes Telegram retry the same update forever

Sparkline in Unicode blocks

the only chart a chat window can render inline

`/predict rice phnom penh`

Rice (mixed, low quality)

Phnom Penh, Phnom Penh · Wholesale

Last recorded (2022-06): \$0.440 / KG

Next period forecast: \$0.461

▲ +4.8%



2020-05 → 2022-06, 24 observations

What this model *cannot do*

One period ahead, and no further

Every feature is a recent price. Feed the model its own output and error compounds fast — there is no multi-step forecast here.

It cannot see shocks coming

Fuel costs, export bans, floods, festival demand. None are in the feature set, and they are exactly what moves prices sharply.

Stale series forecast from stale prices

A series last reported in 2022 gets a 2022-shaped answer. Honest, but not useful for today.

No uncertainty interval

A single number, not a range. The dashed line signals doubt visually, but does not quantify it.

IF I KEPT GOING

Three things

I would add next

Prediction intervals

Quantile regression at 10/90 to turn the dashed line into a shaded band — the honest version of the same signal.

Exogenous features

Fuel price, rainfall, and the lunar calendar for festival demand. The categorical features contribute two percent; these might not.

Retraining on a schedule

The export script already runs headless. A monthly job against the live HDX feed would keep every series current.

THANK YOU

Questions *welcome*

DATA

WFP Global Food Prices — Cambodia

MODEL

XGBoost · 400 trees · scikit-learn pipeline

RUNTIME

Nuxt 4 · Nitro · TypeScript tree traversal

SURFACES

Web app · Telegram bot · one shared engine